

# Firewall Implementation

The firewall will control which network traffic is permitted to reach the testing environment.

The initial firewall rule will demonstrate the principle of **least privilege** by allowing only the web traffic required by the penetration-testing framework.

The primary target for this rule will be OWASP Juice Shop.

## Target

Source:

172.28.0.10  
(pentest-tool)

Destination:

172.28.0.20  
(juice-shop)

Protocol:

TCP

Destination Port:

3000

Action:

ALLOW

This is meant to allow the framework to communicate with OWASP Juice Shop

Using TCP port 3000, as that is the port the application listens on. Traffic that does not match the authorized rule should be denied according to the firewall's default-deny policy. A similar rule should be in place for DVWA.

# Firewall Testing

The firewall implementation will be tested using both authorized and unauthorized traffic.

## Authorized test

From the penetration-testing container:

```
curl http://juice-shop:3000
```

Expected result:

Connection allowed

HTTP response received

## Unauthorized test

A connection attempt to a service or port that has not been authorized should be rejected or dropped.

# IDS Configuration

The IDS will monitor traffic associated with the testing network:

Network:

172.28.0.0/24

The initial detection rule will identify an example HTTP request associated with a basic SQL-injection testing pattern.

A simplified demonstration rule could be:

```
alert http any any -> 172.28.0.20 3000 (  
    msg:"WEB-APP Possible SQL Injection Test";  
    flow:to_server,established;  
    http.uri;  
    Content:" ' ";  
    sid:1000001;  
    rev:1;  
)
```

It alerts when an HTTP request to Juice Shop contains a single quote character in the URI.

# IDS Alert Example

A representative IDS event could look like this:

[\*\*] WEB-APP Possible SQL Injection Test [\*\*]

Source:

172.28.0.10

Destination:

172.28.0.20:3000

Protocol:

TCP / HTTP

Detection:

Single quote detected in HTTP URI

Rule ID:

1000001

Action:

ALERT

Timestamp:

2026-XX-XX XX:XX:XX

The actual timestamp and event fields will be collected from the IDS during implementation.

## IPS Configuration

The demonstration IPS rule will use the same controlled SQL-injection test pattern but will take a blocking action.

Example:

```
drop http any any -> 172.28.0.20 3000 (  
    msg:"WEB-APP Blocked SQL Injection Test";  
    flow:to_server,established;  
    http.uri;  
    content:"";  
    sid:1000002;  
    rev:1;  
)
```

## Example Detected Event

### Event Scenario

The Python framework performs an authorized SQL-injection detection test against Juice Shop.

Example:

Source:

172.28.0.10

Destination:

172.28.0.20:3000

Protocol:

HTTP over TCP

Test Category:

SQL Injection

Detection:

Configured IDS/IPS rule matched suspicious URI content

IDS Response:

Alert generated

IPS Response:

Traffic dropped